

LibraryCard — Written Questions

1. **Which C++ feature did you use to meet B4 (the number never changes)? Why does that choice force you to set the number in a particular place in the constructor?**

I declared the member as `const int number` in `LibraryCard.h`. A `const` data member cannot be assigned after the object is created, so the only chance to give it a value is in the constructor's member-initializer list (I used `number(nextNumber++)` there); assigning it in the constructor body would not compile. This makes the compiler, not my discipline, enforce that nothing ever changes the number.

2. **How did you meet B5 (each rule written once)? Name the feature or features you used.**

Each rule lives in its own *private static member function*: `isValidName()` holds the “at least 2 characters” rule and `cleanFines()` holds the “negative becomes 0” rule. The constructor calls both in its member-initializer list, and `setHolder()` calls `isValidName()`. In addition, *delegating constructors* route the one-argument and default constructors through the two-argument constructor, so all three creation paths share a single body. No check is copy-pasted.

3. **The test program's output ends with five closed lines. Explain why they come out in that order, and why Card 5004 closed appears much earlier.**

Automatic (local) objects are destroyed in *reverse order of construction* when the block they live in ends. `temp` (card 5004) is declared inside its own inner block, so its destructor runs at that block's closing brace, which is why “Card 5004 closed” appears right after the second `Active: 4` line. The five locals in `main` were constructed in the order `ana` (5001), `ben` (5002), `x` (5003), `lobby[0]` (5005), `lobby[1]` (5006), so they are destroyed in exactly the reverse order: `lobby` first, array elements in reverse index order (5006, 5005), then `x` (5003), `ben` (5002), and `ana` (5001).

4. **If `printReport` failed to compile at some point, what was the error and what did you change? If it never failed, what did you do in advance so it wouldn't?**

It never failed. In advance, I marked every member function that only reads the card — the four getters and `print()` — as `const` in the class declaration. The parameter of `printReport` is `const LibraryCard&`, and a `const` object may only call `const` member functions, so without those `const` qualifiers the call `card.print()` would not have compiled.

5. **If copying were allowed, what would go wrong with the card numbers? Use the words `copy constructor` in your answer.**

The compiler-generated copy constructor would copy the `number` field verbatim into the new object, so two live card objects would report the same card number. That breaks the uniqueness rule (B3): after `LibraryCard copy = ana`; both cards would claim to be card 5001, and there would be no way to tell which physical card a record belongs to.

6. **Your class has both constructors and a setter for the holder's name. Why isn't `setHolder` enough on its own?**

A member function can only run on an object that already exists, so the object must be created in a valid state before `setHolder` could ever be called: creation must sanitize the name, assign the card number, and zero the book count without any chance of failure (B2), whereas `setHolder` is allowed to reject a name and return `false`. The card number makes this impossible anyway, because it is a `const` member that no setter can ever assign — only a constructor's initializer list can give it its value.

7. **Suppose you forgot to add `LibraryCard.cpp` to `add_executable`. Which tool reports the problem (compiler or linker), and what does the error say?**

The *linker* reports it. `main.cpp` still compiles, because the declarations in `LibraryCard.h` are enough for the compiler; but with no definitions anywhere, linking fails with errors such as:

```
undefined reference to 'LibraryCard::LibraryCard(std::__cxx11::basic_string<char, std::char_
undefined reference to 'LibraryCard::activeCards()'
undefined reference to 'LibraryCard::count'
collect2: error: ld returned 1 exit status
```

E1 — First error line for each must-not-compile check

Each line was uncommented one at a time and built; the first error line from each build is copied below (line numbers refer to my `main.cpp`).

1. `LibraryCard copy = ana;`

```
main.cpp:58:24: error: use of deleted function 'LibraryCard::LibraryCard(const LibraryCard&)
```

2. `lobby[0] = ana;`

```
main.cpp:59:16: error: use of deleted function 'LibraryCard& LibraryCard::operator=(const Li
```

3. `std::string name = "Zoe"; printReport(name);`

```
main.cpp:60:44: error: invalid initialization of reference of type 'const LibraryCard&'
from expression of type 'std::string' {aka 'std::__cxx11::basic_string<char>'}

```

The third error occurs because the `LibraryCard(std::string)` constructor is `explicit`, so a `std::string` cannot implicitly convert to a `LibraryCard` when initializing the `const LibraryCard&` parameter (B6).